

Laboration 1

AI-Integrerad Spring Boot Service

Introduktion

I denna laboration ska du bygga en middleware-tjänst i Spring Boot. Applikationen fungerar som en bro mellan en slutanvändare och en Large Language Model (LLM). Istället för att användaren pratar direkt med AI:n, går anropet genom din applikation där du kan styra **personlighet (system prompt)**, hantera **kontext (minne)** och säkra upp kommunikationen.

1. Arkitektur

Applikationen ska exponera ett REST-endpoint: `POST /api/v1/chat`.

- Request body ska innehålla användarens fråga och ett val av "personlighet" (t.ex. `helper`, `pirate`, `coder`).

```
{
  "personality": "coder",
  "message": "Hur skriver jag en for-loop i Java?",
  "sessionId": "user-123-abc"
}
```

Fält	Typ	Beskrivning	Exemplvärden	Obligatorisk
personality	String	Bestämmer vilken system-prompt som ska användas.	helper, pirate, coder	Ja
message	String	Själva frågan från slutanvändaren.	"Vad är polymorfism?"	Ja
sessionId	String	Används för att hålla isär olika chatt-historiker (minne).	"uuid-string-123"	Nej Används för att kunna fortsätta en konversation.

- Applikationen mappar valet av personlighet till en specifik **System Prompt**.

- **RestClient:** Använd Spring `RestClient` för att skicka en begäran till den externa modellen (OpenRouter eller LM Studio). [REST Clients :: Spring Framework](#)

2. Minne och Kontext

För att AI:n ska förstå uppföljningsfrågor måste du skicka med tidigare historik i anropet.

- Implementera en enkel in-memory lagring (t.ex. en `Map` eller `List`) som håller reda på de senaste anropen per session eller användare.

3. Säkerhet och Konfiguration (VG)

- **API-nycklar:** Inga nycklar får hårdkodas. Använd miljövariabler (`.env` filer eller systemvariabler) som läses in via `@Value` eller `@ConfigurationProperties`.
- **OpenAPI/Swagger:** Implementera `springdoc-openapi` så att ditt eget API dokumenteras automatiskt på `/swagger-ui.html`.

4. Resiliens och Felhantering (VG)

Nätverksanrop mot AI-modeller kan ofta misslyckas eller ta lång tid.

- **Retry & Backoff:** Om API-anropet misslyckas (t.ex. statuskod 429 eller 503), ska applikationen automatiskt försöka igen med en exponentiell fördröjning.
- **Global Exception Handler:** Skapa en `@ControllerAdvice` som snyggt returnerar felmeddelanden till användaren om AI-tjänsten är nere. Hur hanterar vi fel från ett rest api? [Best Practices for REST API Error Handling | Baeldung](#)

5. Testning

Hur skriver vi tester för vår applikation? Vi vill kunna testa våra rest endpoints och då kan vi mocka vår service klass som anropar externt api alternativt använda ett mockat api som vi kan anropa. Underlättar testning av tex retry då vi kan generera fel på ett förutsägbart sätt för testet.

[WireMock Java - API Mocking for Java and JVM | WireMock](#)